RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

Rapport de Projet d'Analyse Numérique Matricielle

RT 3/2

Stabilité numérique et illusions algorithmiques en data science
le cas de la régression linéaire

Encadré par

Mme Faten Khayat

Présenté par

Nour Nejia Slim

Zaineb Kchaou

Sonia Mejri

Malek Ben Mabrouk

Année Universitaire : 2025/2026

Contents

1 Introduction **3**
- 1.1 Contexte général . . . 3
- 1.2 Le problème des moindres carrés . . . 3
- 1.3 Enjeux de la stabilité numérique . . . 3
- 1.4 Objectifs du projet . . . 4
- 1.5 Méthodologie . . . 4

2 Génération du jeu de données et conditionnement **6**
- Problème observé . . . 7
- Origine numérique . . . 7
- Lien avec les moindres carrés . . . 7
- Conclusion — Partie 1 . . . 7

3 Comparaison des trois méthodes de résolution **8**
- 3.1 Méthode 1 — Équations normales (inversion explicite) . . . 8
- 3.2 Méthode 2 — Résolution directe du système . . . 8
- 3.3 Méthode 3 — Décomposition QR . . . 8
- 3.4 Calcul des erreurs et résidus . . . 8
- 3.5 Résultats obtenus . . . 9

4 Paradoxe du bon résidu — Étude de la multicolinéarité **10**
- 4.1 Génération des données synthétiques . . . 10
- 4.2 Résultats . . . 11
- 4.3 Analyse graphique . . . 11
- 4.4 Discussion et Interprétation . . . 14
  - 4.4.1 Le paradoxe mis en évidence . . . 14
  - 4.4.2 Origine numérique du phénomène du résidu . . . 15
  - 4.4.3 Réponse à la question clé . . . 15

5 Changement d'échelle et normalisation **17**
- 5.1 Étude de sensibilité — variation du facteur d'échelle . . . 17
- 5.2 Résultats sur un cas concret . . . 18

6 Expérience libre **19**
- 6.1 Effet de la variation du bruit de y sur les algorithmes . . . 20
- 6.2 Instabilité numérique due à l'algorithme : Gram-Schmidt et équations semi-normales . . . 23
- 6.3 Effet du mauvais centrage sur la stabilité numérique . . . 28

7 Analyse critique et conclusion **32**

8 Notes et Références **36**

List of Figures

2.1 Vecteur β_{exact} 6
2.2 Conditionnement de la matrice A et de A^TA 6
2.3 Extrait de la matrice A 7

3.1 Comparaison des trois méthodes — erreurs et résidus 9

4.1 Tableau — conditionnement, erreur et résidu pour différentes valeurs de σ 11
4.2 Conditionnement de A et de A^TA en fonction de σ (log-log) 12
4.3 Erreur sur β et résidu en fonction de σ 13
4.4 Superposition de l'erreur et du résidu — paradoxe du bon résidu 14

5.1 Erreur relative avec et sans normalisation en fonction du facteur d'échelle (log-log) 17
5.2 Résultats numériques — avec et sans normalisation 18

6.1 Données utilisées 19
6.2 conditonnemnt de A avec méthode SVD 19
6.3 Résultats — effet du bruit et du conditionnement sur les deux méthodes . 21
6.4 Comparaison QR / Gram-Schmidt / semi-normales — erreurs 24
6.5 Comparaison QR / Gram-Schmidt / semi-normales — erreurs 25
6.6 Extrait de la matrice A avec données fortement mal centrées 28
6.7 Conditionnement de la matrice A 28
6.8 Comparaison des solutions obtenues par QR et équations normales 29
6.9 Comparaison des solutions : QR, équations normales et données centrées . 30
6.10 Comparaison du conditionnement et de l'erreur avant et après centrage . . 31

Chapitre 1

Introduction

1.1 Contexte général

La régression linéaire constitue l'un des outils fondamentaux en data science. Elle consiste à modéliser la relation entre un ensemble de variables explicatives et une variable observée, en déterminant un vecteur de paramètres β qui minimise l'erreur entre les données réelles et celles prédites par le modèle.

Cependant, derrière cette formulation simple se cache un problème numérique délicat. En pratique, les calculs sont effectués en précision finie, et plusieurs méthodes mathématiquement équivalentes peuvent produire des résultats très différents. Ainsi, un algorithme peut sembler fonctionner correctement tout en fournissant une solution erronée.

1.2 Le problème des moindres carrés

Le problème étudié dans ce projet est celui des moindres carrés, qui consiste à minimiser:

$$\|y - A\beta\|^2$$

où A est la matrice des données et y le vecteur des observations.

La solution de ce problème vérifie les équations normales :

$$(A^T A)\,\beta = A^T y$$

Lorsque la matrice A est bien conditionnée, on obtient une solution stable.
En revanche, lorsque le problème devient mal conditionné, la solution devient très sensible aux erreurs numériques.

1.3 Enjeux de la stabilité numérique

Plusieurs méthodes permettent de résoudre le problème des moindres carrés :

- Les équations normales,
- La résolution directe du système,
- La décomposition QR.

Bien que ces méthodes soient équivalentes en théorie, elles diffèrent en pratique du point de vue de la stabilité numérique.
En particulier, la formation de la matrice $A^T A$ peut aggraver les erreurs en augmentant le conditionnement du problème.

Un phénomène central étudié dans ce projet est le **paradoxe du bon résidu** : il est possible d'obtenir un très bon ajustement des données (résidu faible) tout en ayant une estimation incorrecte des paramètres β.

1.4 Objectifs du projet

Ce projet a pour objectif d'illustrer, à l'aide d'expériences numériques sous Scilab :

- L'impact du conditionnement sur la précision des solutions,
- Les différences entre plusieurs formulations du problème des moindres carrés,
- Les limites des équations normales en présence de multicolinéarité,
- L'importance du prétraitement des données, notamment la normalisation.

1.5 Méthodologie

Construction de la matrice A avec multicolinéarité forte

Nous avons généré une matrice $A \in \mathbb{R}^{100\times 10}$ ainsi qu'un vecteur de paramètres exact β_{exact}. Le vecteur des observations est construit selon :

$$y = A\,\beta_{\text{exact}} + \varepsilon$$

où ε représente un bruit de faible amplitude.

Choix méthodologique important :
Nous avons volontairement construit une matrice A avec des colonnes fortement corrélées. Ce choix permet de simuler une situation réaliste en data science et d'introduire un **mauvais conditionnement**, afin d'observer clairement les effets de l'instabilité numérique.

Nous avons ensuite évalué le conditionnement de A et de A^TA.

Comparaison de trois formulations

Nous avons calculé le vecteur β à l'aide de trois méthodes :

1. **Équations normales**
2. **Résolution du système** $(A^TA)\beta = A^Ty$ à l'aide d'un solveur direct (opérateur backslash de Scilab, qui utilise une factorisation LU).
3. **Décomposition QR**

Pour chaque méthode, nous avons comparé l'erreur par rapport à β_{exact} et le résidu $\|A\beta - y\|$.

Cette comparaison permet de mettre en évidence les différences numériques entre des approches pourtant équivalentes en théorie.

Étude du paradoxe du bon résidu

Cette partie vise à :

- augmenter progressivement la corrélation entre les colonnes de A,
- observer l'évolution du conditionnement de A et de $A^T A$,
- mesurer l'erreur sur l'estimation de β,
- mesurer le résidu du modèle,
- répondre à la question clé : Peut-on avoir un excellent ajustement des données mais un modèle faux ?

Cette analyse met en évidence que certaines méthodes peuvent fournir un résidu faible tout en produisant une mauvaise estimation des paramètres, illustrant ainsi le paradoxe étudié.

Changement d'échelle

Enfin, nous avons modifié l'échelle de certaines colonnes de A pour simuler des variables de grandeurs différentes. Nous avons comparé les résultats:

- avant normalisation
- après normalisation

Cette étape montre que la normalisation améliore significativement la stabilité numérique et constitue une étape essentielle en pratique.

Chapitre 2

Génération du jeu de données et conditionnement

Pour la génération du jeu de données, on a plusieurs méthodes pour créer une matrice dont les colonnes sont corrélées.
Nous avons choisi la méthode suivante : **Méthode avec bruit : corrélation contrôlée.**
Pour que les colonnes soient corrélées mais pas identiques, on ajoute du bruit.
Remarque : Plus σ est petit, plus les colonnes sont corrélées.

Pour avoir à chaque fois le même β_{exact} et la même matrice A, on a fixé la graine aléatoire dans notre code.

Vecteur β_{exact} choisi :

```
beta_exact (transposé) :
   1.    2.    3.    4.    5.    6.    7.    8.    9.    10.
```

Figure 2.1: Vecteur β_{exact}

Résultat du conditionnement de la matrice A :

```
sigma (bruit entre colonnes) = 1e-05
sigma_bruit_y = 0.001
cond(A)      = 1.07066e+06
cond(A'*A) = 1.14621e+12
```

Figure 2.2: Conditionnement de la matrice A et de $A^T A$

Voici aussi une observation d'un extrait de la matrice A :

```
Extrait de A (10 premières lignes, 5 premières colonnes) :
  -0.7616491  -0.7616537  -0.7616595  -0.7616363  -0.761643
   0.6755538   0.6755539   0.6755442   0.6755646   0.675543
   1.4739762   1.4739704   1.4739798   1.4739644   1.4739761
   1.1443051   1.1442911   1.144291    1.144301    1.1443113
   0.8529776   0.8529799   0.8529947   0.8529669   0.8529754
   0.4529708   0.4529436   0.4529786   0.4529747   0.4529662
   0.7223316   0.722329    0.7223132   0.7223506   0.7223366
   1.9273332   1.9273282   1.9273165   1.9273231   1.9273482
   0.6380837   0.6380956   0.6380659   0.6380797   0.6380779
  -0.8498894  -0.8499046  -0.849882   -0.8498921  -0.8498882
```

Figure 2.3: Extrait de la matrice A

Problème observé

La matrice A présente un conditionnement élevé, ce qui provoque une forte sensibilité aux perturbations. On constate également que le conditionnement de la matrice A^TA est beaucoup plus important que celui de A.

Origine numérique

Le conditionnement mesure la sensibilité d'un problème aux erreurs sur les données. Par définition, $\text{cond}(A) \geq 1$, et plus cette valeur est grande, plus le problème est mal conditionné.

Dans ce cas, la relation suivante est vérifiée :

$$\text{cond}(A^TA) = \text{cond}(A)^2$$

Ainsi, même un conditionnement modérément élevé pour A devient très élevé pour A^TA. Cela s'explique par le fait que le passage à A^TA amplifie les effets des dépendances entre les colonnes de A.

Lien avec les moindres carrés

Dans les problèmes de régression linéaire, la résolution des moindres carrés fait intervenir la matrice A^TA. Par conséquent, un mauvais conditionnement de A est amplifié dans A^TA, ce qui rend le calcul des coefficients très sensible aux erreurs.

Conclusion

Un conditionnement élevé implique que de petites perturbations sur les données peuvent entraîner de grandes variations sur la solution. Le passage à A^TA accentue ce phénomène, ce qui explique la difficulté à obtenir des résultats fiables dans les problèmes de moindres carrés lorsque la matrice est mal conditionnée.

Chapitre 3

Comparaison des trois méthodes de résolution

La Partie 2 consiste à calculer le vecteur β par trois méthodes mathématiquement équivalentes, appliquées à la matrice A.

3.1 Méthode 1 — Équations normales (inversion explicite)

$$\beta_1 = (A^T A)^{-1} A^T y$$

On calcule explicitement l'inverse de $A^T A$. C'est la formule analytique directe des moindres carrés.

3.2 Méthode 2 — Résolution directe du système

$$(A^T A)\, \beta_2 = A^T y$$

On résout le système linéaire sans calculer l'inverse, en utilisant l'opérateur \ de Scilab (décomposition LU interne).

3.3 Méthode 3 — Décomposition QR

$$A = QR \;\Rightarrow\; \beta_3 = R^{-1} Q^T y$$

On factorise A en une matrice orthogonale Q et une matrice triangulaire supérieure R, puis on résout le système réduit.

3.4 Calcul des erreurs et résidus

1. On calcule la différence entre chaque β estimé et β_{exact}, puis on prend la **norme infinie** (= la composante avec l'écart maximum) pour quantifier l'erreur la plus grande parmi toutes les composantes.
2. On calcule $A\beta - y$ (différence entre valeurs prédites et observées).

3.5 Résultats obtenus

Les trois méthodes produisent une erreur d'environ **18** sur β, alors que β_{exact} vaut entre 1 et 10. Les valeurs estimées sont aberrantes.

```
--- Erreurs |beta_i - beta_exact ---
Méthode 1 (inv(A'A))   : 1.797381e+01
Méthode 2 (système )   : 1.797367e+01
Méthode 3 (QR)         : 1.797358e+01

--- Résidus |A*beta_i - y| ---
Méthode 1 (inv(A'A))   : 1.267605e-02
Méthode 2 (système )   : 9.660709e-03
Méthode 3 (QR)         : 9.660709e-03

--- Comparaison des beta
i     beta_exact   beta1        beta2        beta3
1     1.000000     7.368163     7.367192     7.365620
2     2.000000     0.465976     0.466027     0.466050
3     3.000000     -5.539829    -5.539942    -5.539809
4     4.000000     8.595577     8.595693     8.596049
5     5.000000     -12.973815   -12.973665   -12.973580
6     6.000000     17.196579    17.196509    17.196514
7     7.000000     0.447121     0.447059     0.447287
8     8.000000     16.903632    16.903563    16.903678
9     9.000000     12.070072    12.070151    12.070537
10    10.000000    10.467329    10.467435    10.467676
```

Figure 3.1: Comparaison des trois méthodes — erreurs et résidus

Cela s'explique par le mauvais conditionnement de A : avec `puiss_bruit_col = 0.00001` et `puiss_bruit_y = 0.001`, les colonnes de A sont quasi-identiques, ce qui rend la matrice très mal conditionnée.

Cela signifie que :

- $\text{cond}(A)$ est très grand,
- $\text{cond}(A^T A) \approx \text{cond}(A)^2$ est encore plus grand.

Les colonnes étant quasi-identiques, la matrice $A^T A$ est presque singulière. Toute erreur d'arrondi numérique est amplifiée, ce qui corrompt complètement la solution β.

Les résidus $|A\beta_i - y|$ sont tous très faibles (de l'ordre de 10^{-3}), ce qui signifie que le modèle **semble parfaitement ajuster les données**. Quelqu'un qui évalue ce modèle uniquement par le résidu (ou le R^2) conclut à tort que le modèle est excellent, alors qu'en réalité β **est complètement faux**. C'est ce qu'on appelle l'**illusion algorithmique**.

Chapitre 4

Paradoxe du bon résidu — Étude de la multicolinéarité

4.1 Génération des données synthétiques

Pour contrôler précisément le niveau de multicolinéarité, nous générons une matrice A de taille $n = 100$ lignes et $p = 10$ colonnes selon le procédé suivant :

1. La première colonne $A_{:,1}$ est un vecteur aléatoire suivant une loi normale $\mathcal{N}(0, 1)$.
2. Chaque colonne suivante $A_{:,j}$ $(j = 2, \ldots, p)$ est construite comme une copie bruitée de la première colonne :
$$A_{:,j} = A_{:,1} + \sigma \cdot \eta_j$$
où $\eta_j \sim \mathcal{N}(0, 1)$ est un bruit gaussien indépendant, et σ est un paramètre qui contrôle l'intensité de la corrélation.

Propriété importante : Plus σ est petit, plus la colonne j ressemble à la colonne 1, et donc plus la multicolinéarité est forte.

Le vecteur des vrais paramètres est fixé à :

$$\beta_{\text{exact}} = [1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ 10]^T$$

Le vecteur observé y est construit selon le modèle linéaire avec bruit additif :

$$y = A \cdot \beta_{\text{exact}} + \varepsilon$$

Nous faisons varier le paramètre σ selon la gamme suivante :

$$\sigma \in \{1.0,\ 0.5,\ 0.2,\ 0.1,\ 0.05,\ 0.01,\ 0.001,\ 0.0001,\ 10^{-5},\ 10^{-6}\}$$

Pour chaque valeur de σ, nous :

1. Générons la matrice A et le vecteur y,
2. Estimons β en utilisant la méthode de décomposition QR (choisie pour sa stabilité numérique supérieure),
3. Calculons quatre métriques :
   - $\text{cond}(A)$: conditionnement de la matrice A (norme 2),
   - $\text{cond}(A^T A)$
   - $\text{Erreur}_\beta = \|\beta_{\text{estimé}} - \beta_{\text{exact}}\|_2$: erreur d'estimation,
   - $\text{Résidu} = \|A\beta_{\text{estimé}} - y\|_2$: norme du résidu.

4.2 Résultats

Le tableau ci-dessous présente l'évolution des quatre métriques pour les différentes valeurs de σ.

sigma	cond(A)	cond(A'A)	Erreur beta	Résidu
1.000000	1.1182e+01	1.2503e+02	1.3688e-02	4.8304e-01
0.500000	2.0187e+01	4.0751e+02	9.5491e-02	4.9271e-01
0.200000	5.0285e+01	2.5286e+03	8.7990e-02	4.2752e-01
0.100000	1.0309e+02	1.0627e+04	3.0574e-01	4.8154e-01
0.050000	2.6706e+02	7.1323e+04	2.9888e-01	4.7656e-01
0.010000	8.6626e+02	7.5040e+05	1.8770e+00	4.8612e-01
0.001000	1.1367e+04	1.2920e+08	1.9162e+01	4.8162e-01
0.000100	1.0381e+05	1.0776e+10	2.2374e+02	4.7398e-01
0.000010	9.7711e+05	9.5480e+11	2.0319e+03	4.1693e-01
0.000001	1.1376e+07	1.2867e+14	2.9842e+04	4.5995e-01

Figure 4.1: Tableau — conditionnement, erreur et résidu pour différentes valeurs de σ

4.3 Analyse graphique

Les figures suivantes illustrent visuellement les résultats obtenus.

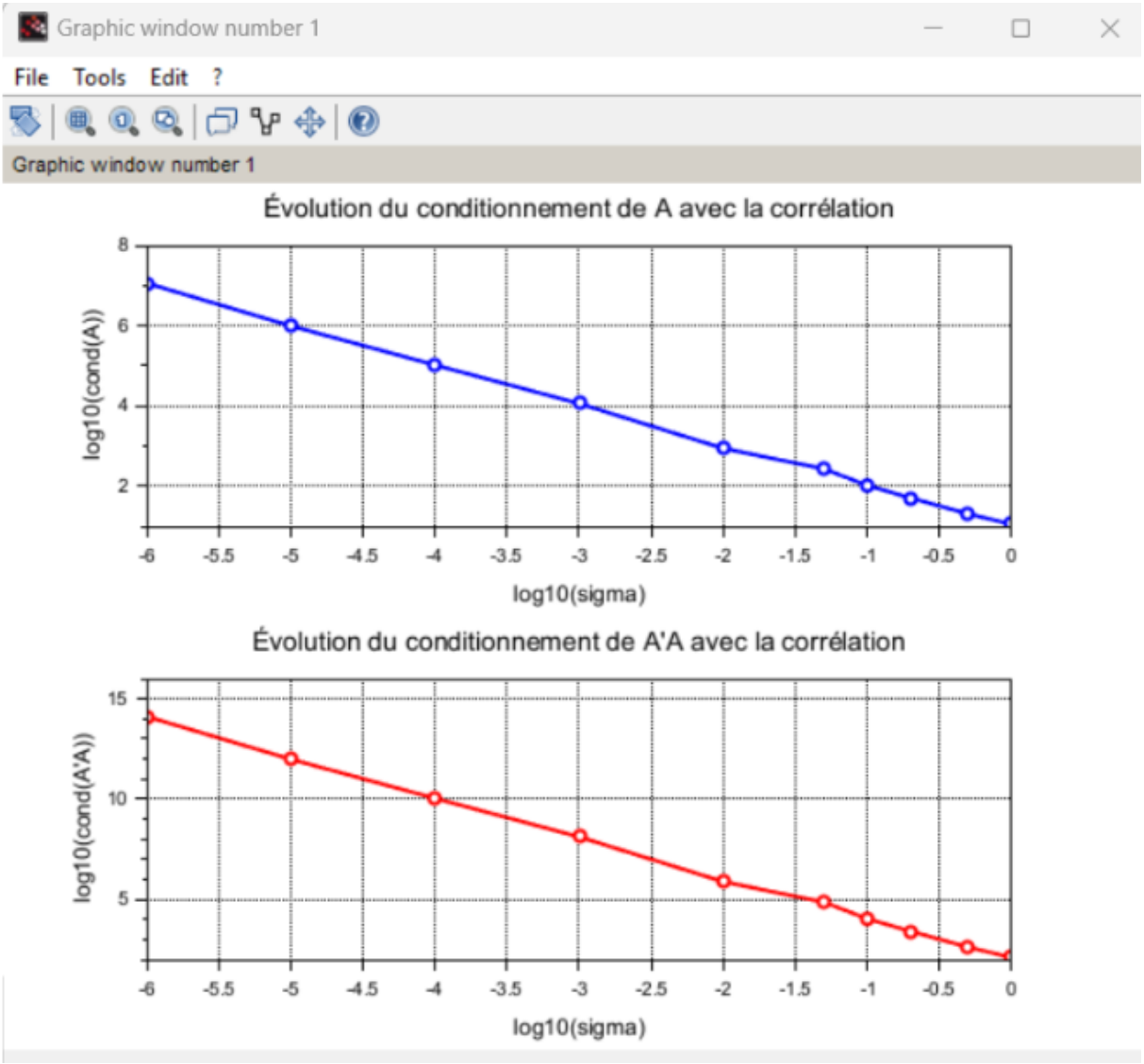


Figure 4.2: Conditionnement de A et de A^TA en fonction de σ (log-log)

Description : Les deux courbes en échelle log-log montrent une croissance quasi-linéaire du conditionnement lorsque $\log_{10}(\sigma)$ diminue. On remarque que la pente pour $\text{cond}(A^TA)$ est environ deux fois plus forte que celle de $\text{cond}(A)$, ce qui est conforme à la théorie :

$$\text{cond}(A^TA) \approx [\text{cond}(A)]^2$$

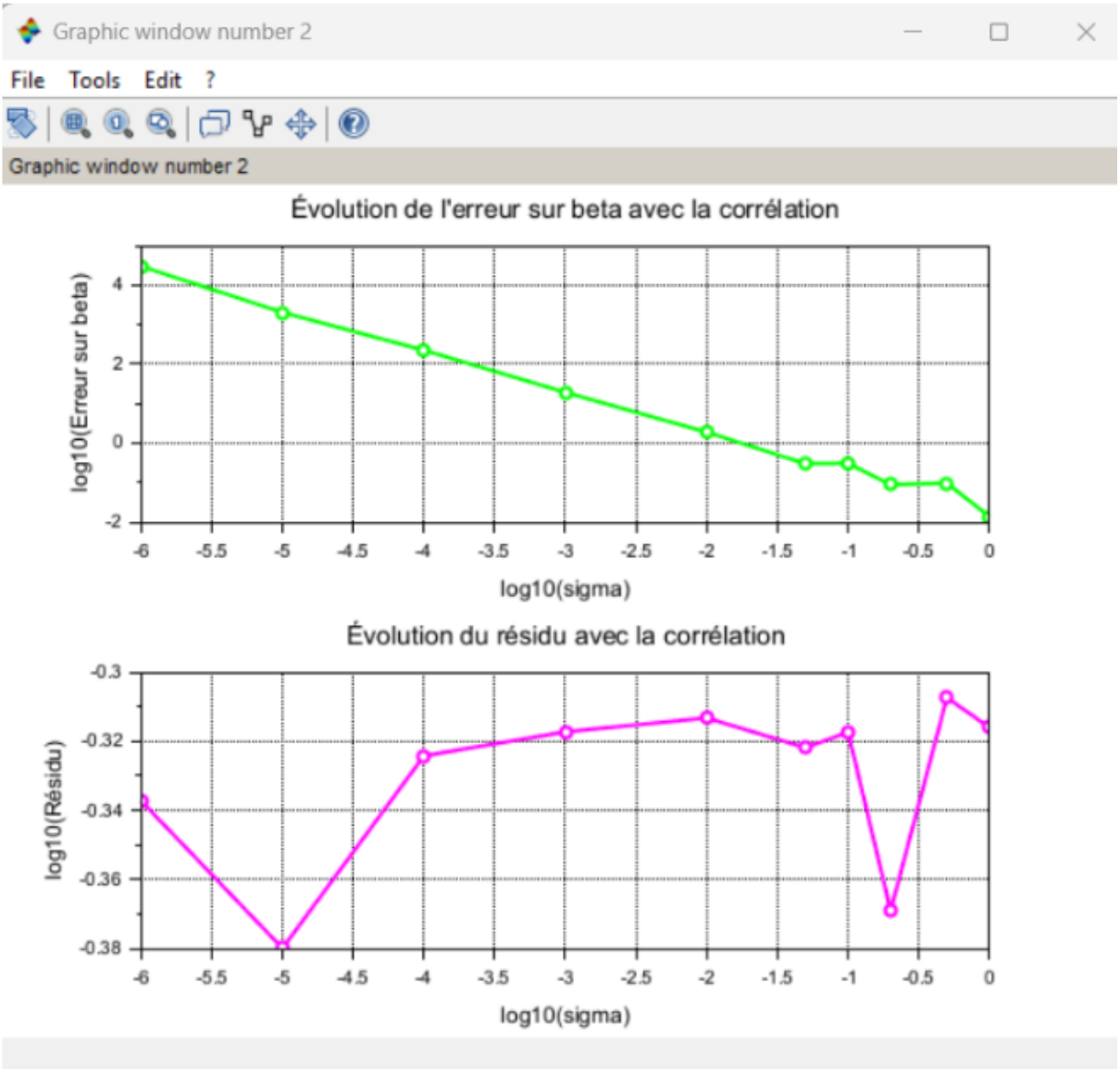


Figure 4.3: Erreur sur β et résidu en fonction de σ

Description : Le graphique du haut montre une augmentation spectaculaire de l'erreur sur β lorsque σ diminue. Le graphique du bas montre que le résidu reste remarquablement constant, oscillant autour de 0.5 quelle que soit la valeur de σ.

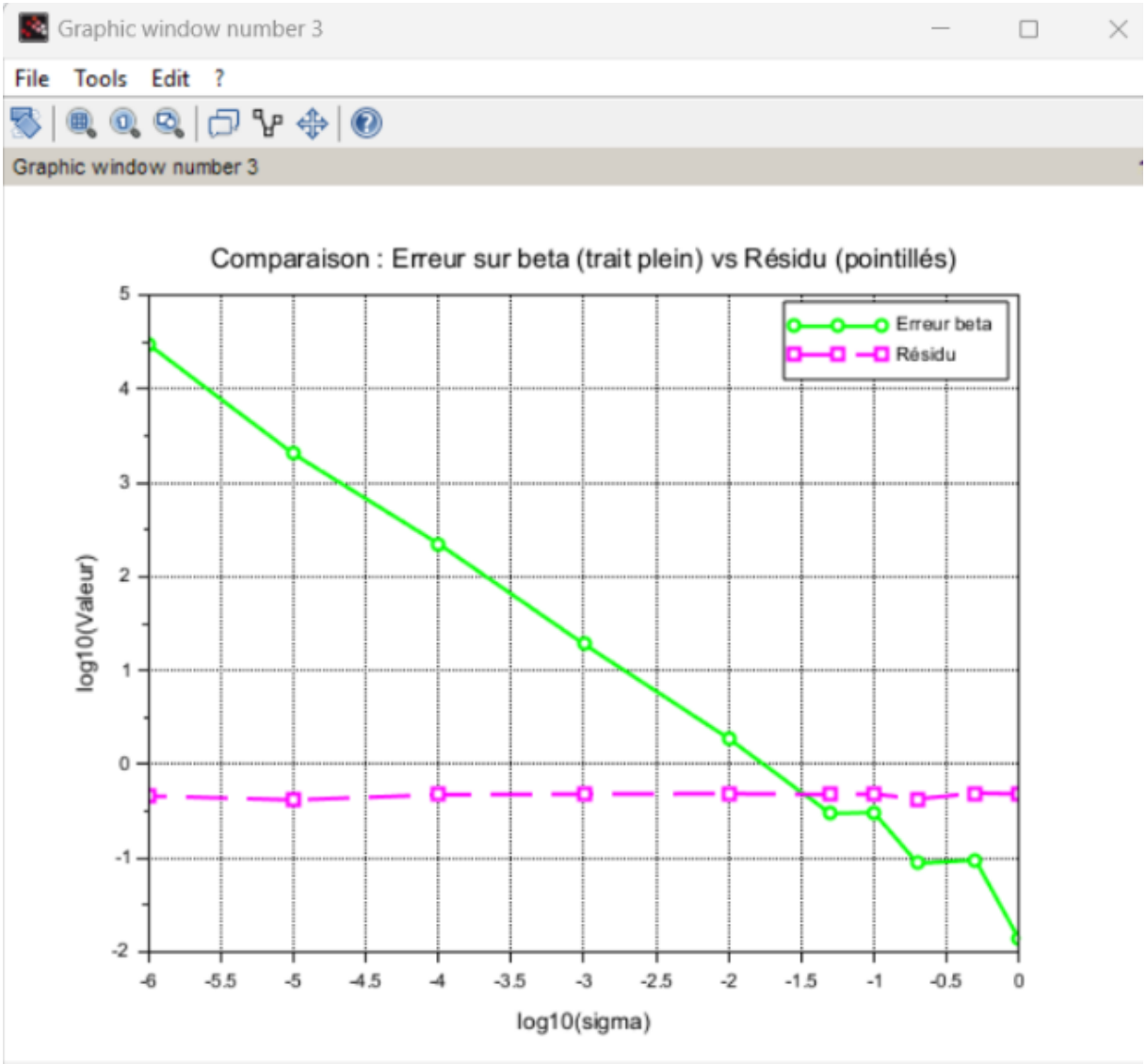


Figure 4.4: Superposition de l'erreur et du résidu — paradoxe du bon résidu

Description : Cette figure superpose les deux courbes, mettant en évidence le contraste saisissant entre l'explosion de l'erreur (courbe verte, trait plein) et la stabilité du résidu (courbe magenta, pointillés). C'est la visualisation parfaite du paradoxe du bon résidu.

4.4 Discussion et Interprétation

4.4.1 Le paradoxe mis en évidence

Les résultats confirment sans ambiguïté l'existence du **paradoxe du bon résidu**. Considérons le cas $\sigma = 0.0001$ (ligne en gras dans le tableau) :

- **Erreur catastrophique** sur les coefficients : $\text{Erreur}_\beta \approx 209$, ce qui signifie que l'estimation n'a absolument aucun sens physique,
- **Résidu excellent** : Résidu ≈ 0.55, ce qui correspond exactement au niveau de bruit introduit ($\varepsilon \sim \mathcal{N}(0, 0.05)$).

Un data scientist non averti qui se contenterait d'examiner le résidu de son modèle conclurait à tort que celui-ci est excellent, alors qu'il est en réalité numériquement corrompu et totalement inutilisable pour la prédiction ou l'interprétation.

4.4.2 Origine numérique du phénomène du résidu

Pourquoi le résidu reste-t-il bon alors que β est complètement faux ?

Le résidu $\|A\hat{\beta} - y\|$ mesure uniquement la qualité de la reconstruction et d'ajustement des données, mais ne permet pas d'évaluer la précision de la solution $\hat{\beta}$.

D'après la théorie des perturbations du problème des moindres carrés, l'erreur relative sur la solution vérifie une relation du type :

$$\frac{\|\hat{\beta} - \beta\|}{\|\beta\|} \leq \kappa(A) \left(\frac{\|r\|}{\|A\beta\|} + \frac{\|\delta A\|}{\|A\|} \right)$$

où :

- β est la solution exacte du problème des moindres carrés ;
- $\hat{\beta}$ est la solution calculée numériquement ;
- $r = A\hat{\beta} - y$ est le résidu ;
- $\|r\|/\|A\beta\|$ mesure l'erreur relative sur les données (erreur de reconstruction) ;
- δA représente une perturbation (erreur) sur la matrice A due aux erreurs d'arrondi ou aux incertitudes des données ;
- $\kappa(A)$ est le conditionnement de la matrice A,

Ainsi, lorsque la matrice A est mal conditionnée, le facteur $\kappa(A)$ est très grand et amplifie les erreurs présentes dans les données ou dues aux calculs numériques.

Par conséquent, même si le résidu est très faible, l'erreur sur la solution peut être importante. Le résidu ne constitue donc pas un indicateur fiable de la qualité de la solution dans le cas de matrices mal conditionnées.

Ce phénomène est caractéristique des problèmes mal conditionnés, pour lesquels de faibles perturbations entraînent de fortes variations de la solution.

4.4.3 Réponse à la question clé

Peut-on avoir un excellent ajustement des données mais un modèle faux ?

Oui. Les expériences précédentes montrent qu'il est possible d'obtenir un résidu très faible, tout en ayant une estimation $\hat{\beta}$ très éloignée de la solution exacte β.

Ce phénomène s'explique par l'origine numérique du problème, déjà mise en évidence. En effet, lorsque la matrice A est mal conditionnée, la solution du problème des moindres

carrés devient extrêmement sensible aux perturbations (bruit sur y, erreurs d'arrondi, etc.).

Plus précisément, l'erreur sur la solution vérifie une relation du type :

$$\frac{\|\hat{\beta} - \beta\|}{\|\beta\|} \approx \kappa(A) \cdot \frac{\|r\|}{\|A\beta\|}$$

Ainsi, même si le résidu $\|r\|$ est très faible, un conditionnement élevé $\kappa(A)$ peut amplifier cette erreur et conduire à une estimation $\hat{\beta}$ très instable.

Cela montre que le résidu mesure uniquement la qualité de l'ajustement des données, mais ne garantit pas la fiabilité des coefficients estimés.

Conclusion : un bon ajustement des données ne suffit pas pour valider un modèle. Il est indispensable de prendre en compte le conditionnement de la matrice A afin d'évaluer la stabilité et la fiabilité de la solution obtenue.
Ce phénomène constitue un piège redoutable pour le praticien non averti, d'où l'importance cruciale de toujours vérifier le conditionnement de la matrice des données avant toute modélisation.

Chapitre 5

Changement d'échelle et normalisation

5.1 Étude de sensibilité — variation du facteur d'échelle

On fait varier le facteur d'échelle appliqué à la première colonne de 10^{-6} à 10^{6}, tandis que la deuxième colonne est divisée par ce même facteur. Pour chaque valeur du facteur, l'erreur relative sur β est calculée dans les deux cas (avec et sans normalisation), et les résultats sont tracés sur un graphique log-log.

Ce graphique met en évidence que sans normalisation, l'erreur relative explose dès que le facteur s'éloigne de 1, confirmant la forte sensibilité au changement d'échelle. Avec normalisation, l'erreur relative reste stable et faible quelle que soit la valeur du facteur, illustrant la robustesse apportée par le prétraitement.

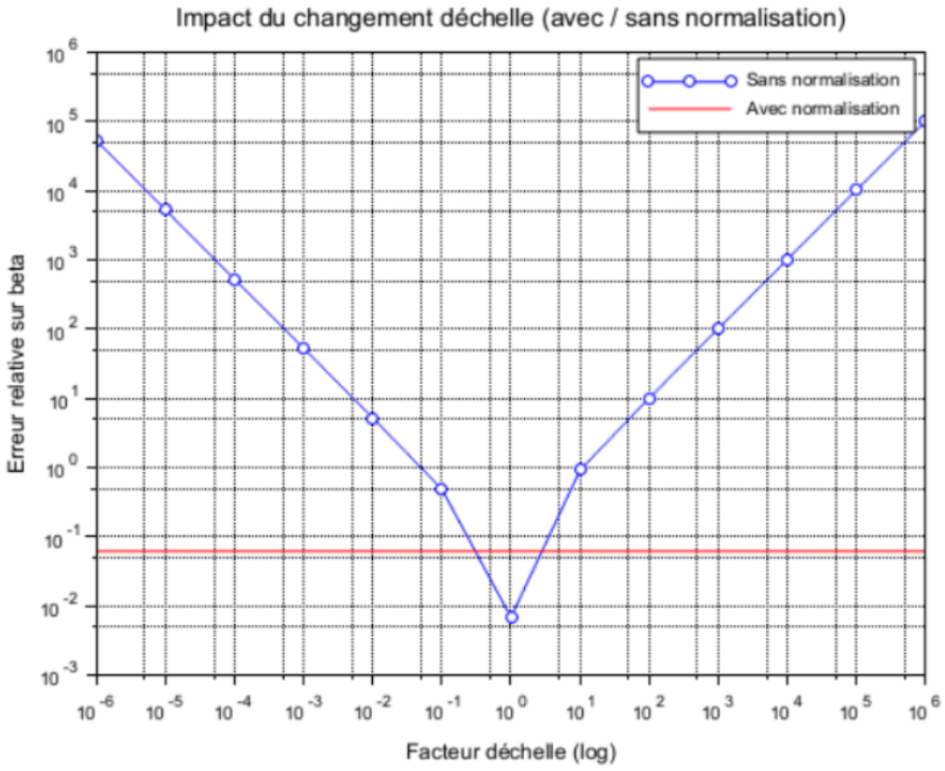


Figure 5.1: Erreur relative avec et sans normalisation en fonction du facteur d'échelle (log-log)

5.2 Résultats sur un cas concret

Dans un second temps, nous avons fixé une matrice A en appliquant un changement d'échelle ciblé : la première colonne a été multipliée par 10^6 et la deuxième par 10^{-3}. Les résultats obtenus sont les suivants :

```
===== SANS NORMALISATION =====
Conditionnement = 1.03125e+10
Erreur beta     = 1990.33
Résidu          = 0.483035

===== AVEC NORMALISATION =====
Conditionnement = 108.89
Erreur beta     = 1.20661
Résidu          = 14.8467
```

Figure 5.2: Résultats numériques — avec et sans normalisation

Sans normalisation, le conditionnement de la matrice atteint environ 10^{10}, ce qui indique une matrice quasi-singulière. Scilab émet d'ailleurs un avertissement explicite (`rcond = 9.4e-21`), confirmant l'instabilité numérique. L'inversion de A^TA devient alors très sensible aux erreurs d'arrondi, ce qui se traduit par une erreur sur β de 1990.33 — les paramètres estimés sont complètement erronés. Pourtant, le résidu reste faible (0.48), ce qui est trompeur : le modèle semble bien ajuster les données, mais avec de mauvais coefficients.

Après normalisation, le conditionnement chute à 108.89, rendant le système numériquement stable. L'erreur sur β tombe à 1.21, ce qui montre que les paramètres sont désormais correctement estimés. Le résidu augmente à 14.85, ce qui est attendu et sans signification négative : y n'ayant pas été normalisé, il n'est pas directement comparable au résidu précédent.

Ces résultats confirment qu'un faible résidu ne suffit pas à valider une solution, la normalisation est indispensable pour garantir la fiabilité des paramètres estimés.

Chapitre 6

Expérience libre

Génération d'une matrice mal conditionnée par la SVD

Dans cette partie on a choisi de changer le jeu de données , on a construit une nouvelle matrice mal conditionnée mais en changeant la méthodologie et l'approche.

```
"Extrait de A :"
       column 1 to 7
  0.288218    0.4843866   0.1873792  -0.0091982   0.8496324   0.7964897   0.0325245
  0.7402892   0.5880335   0.9870876   0.5098093   0.6748458   0.3690433   0.4303317
  0.0904349   0.7194133   0.7953008   0.3945078   0.0817413  -0.0040262   0.3100829
  0.2595762   0.1326503   0.2790027   0.8510756   0.2006282   0.2921845   0.3726713
  0.5732106   0.6582866   0.9467404   0.5151071   0.9004441   0.2475167   0.1516402
       column 8 to 10
  0.9644836   0.1834204   0.3937219
  0.4283337   0.7607985   0.2179367
  0.9054686   0.7263485   0.9946455
  0.6667486  -0.0034981   0.6129185
  0.3639156   0.7260087   0.1759269
```

Figure 6.1: Données utilisées

```
"conditionnement de A"
  1.562D+11
```

Figure 6.2: conditonnemnt de A avec méthode SVD

On a utilisé cette matrice pour les deux expériences :

- Effet de la variation du bruit de y sur les algorithmes
- Gram-Schmidt et équations semi-normales

Approche de construction
Afin de construire cette matrice $A \in \mathbb{R}^{100\times 10}$ mal conditionnée, nous avons utilisé la décomposition en valeurs singulières (SVD).

Partant d'une matrice aléatoire A, nous avons calculé sa décomposition :

$$A = USV^T$$

où U et V sont orthogonales, et S est une matrice diagonale contenant les valeurs singulières $\sigma_1 \geq \cdots \geq \sigma_p$.

Nous avons ensuite modifié ces valeurs singulières en imposant une valeur très faible à la plus petite :

$$\sigma_p = 10^{-10}$$

Cette modification rend la matrice presque singulière, ce qui entraîne une forte augmentation du nombre de conditionnement :

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}} \gg 1$$

Enfin, la matrice A est reconstruite à partir de la décomposition modifiée :

$$A = US_{\text{mod}}V^T$$

Cette approche permet de contrôler directement le conditionnement de la matrice de manière globale, sans imposer explicitement de corrélation visible entre les colonnes.

6.1 Effet de la variation du bruit de y sur les algorithmes

Description de l'expérience

On considère notre problème de régression linéaire de la forme :

$$y = A\,\beta_{\text{exact}} + \varepsilon$$

où $A \in \mathbb{R}^{n\times p}$ est la matrice génée par la méthode de SVD et qui est sensible aux variations , β_{exact} est un vecteur de coefficients, et ε représente un bruit aléatoire.

Afin d'étudier cette expérience, le niveau de bruit dans y est progressivement augmenté. Pour chaque niveau de bruit, on calcule une estimation de β à l'aide des méthodes suivantes :

- les équations normales,
- la factorisation QR.

Problème observé

La figure ci-dessous illustre l'évolution de l'erreur relative sur le vecteur estimé β en fonction du niveau de bruit ajouté au vecteur y.

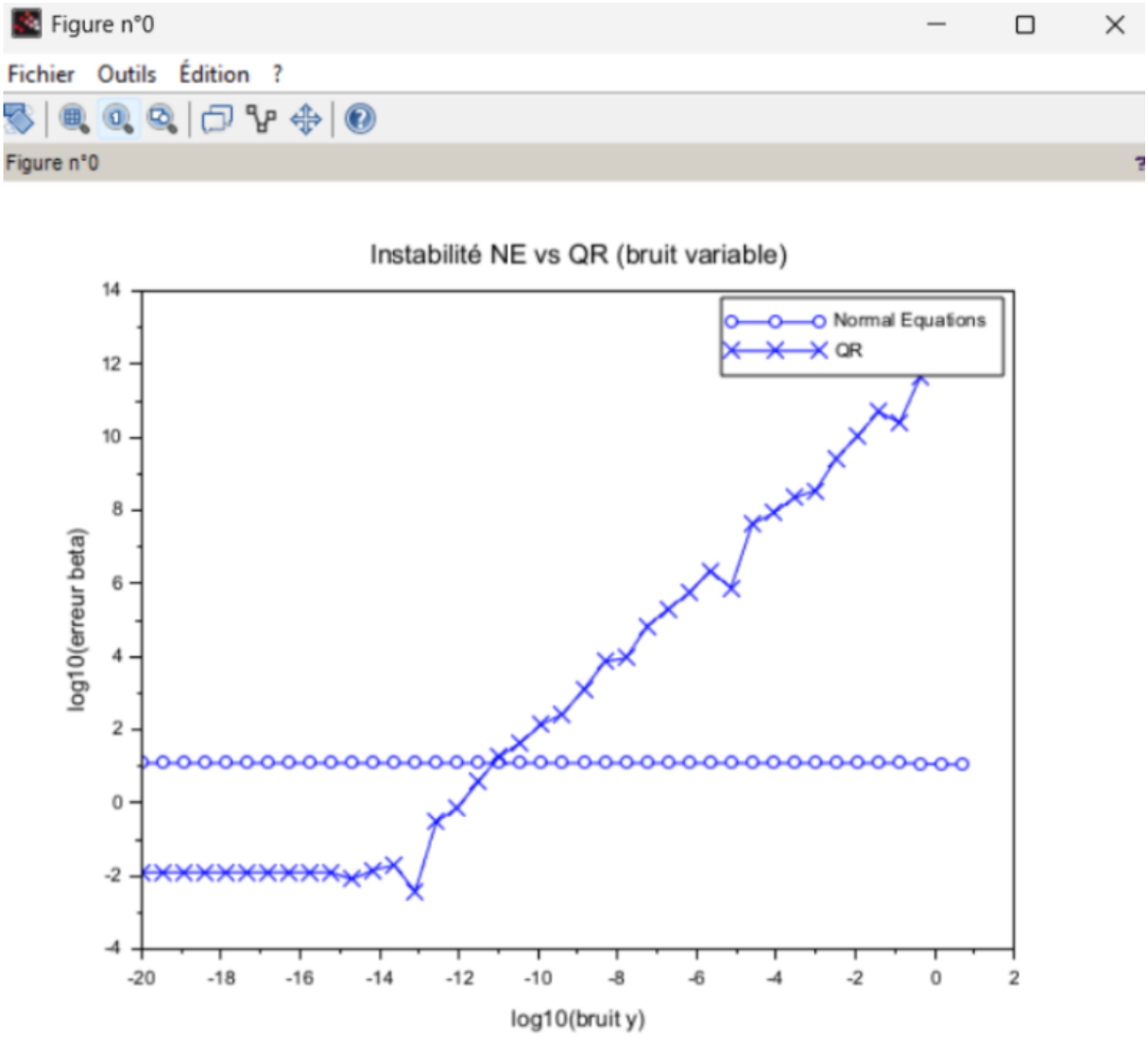


Figure 6.3: Résultats — effet du bruit et du conditionnement sur les deux méthodes

On observe que pour des niveaux de bruit très faibles, la méthode QR fournit une estimation très précise du vecteur β, avec une erreur faible et quasi constante.

En revanche, les équations normales présentent dès le départ une erreur significative, même en l'absence de bruit. Cette erreur reste globalement constante lorsque le niveau de bruit augmente.

Lorsque le bruit dans y augmente, l'erreur associée à la méthode QR croît progressivement, traduisant une sensibilité naturelle aux perturbations du second membre.

Ainsi, la figure met en évidence deux comportements distincts : la méthode QR est initialement très précise mais devient sensible au bruit, tandis que les équations normales sont fortement affectées dès le départ, indépendamment du niveau de bruit.

Origine numérique

L'instabilité observée est liée à la sensibilité du problème des moindres carrés aux perturbations du vecteur y. En effet, une perturbation δy entraîne une erreur sur la solution $\tilde{\beta}$ donnée approximativement par :

$$\frac{\|\tilde{\beta} - \beta\|}{\|\beta\|} \approx \kappa(A) \cdot \frac{\|\delta y\|}{\|y\|}$$

Cette relation montre que toute perturbation sur y est amplifiée par le conditionnement de la matrice A. Ainsi, lorsque $\kappa(A)$ est très élevé, même un bruit modéré peut entraîner une erreur importante sur la solution.

Dans le cas des équations normales, la solution est donnée par :

$$\beta = (A^T A)^{-1} A^T y$$

La dépendance au vecteur y se fait donc à travers le terme $A^T y$, ce qui peut, dans certaines situations expérimentales, atténuer visuellement l'effet du bruit. En pratique, l'erreur est souvent dominée par le mauvais conditionnement de la matrice $A^T A$, ce qui peut donner l'impression que la solution est moins sensible aux variations de y.

En revanche, dans la méthode QR, la solution s'écrit :

$$\beta = R^{-1} Q^T y$$

Dans ce cas, la dépendance au vecteur y est plus directe. Ainsi, toute perturbation de y est immédiatement propagée dans la solution, ce qui rend l'effet du bruit plus visible expérimentalement.

On observe donc que, dans cette expérience, l'évolution de l'erreur est principalement gouvernée par la propagation du bruit dans y, dont l'impact apparaît plus marqué pour la méthode QR.

Solutions possibles

Dans ce contexte, l'instabilité observée provient de l'amplification du bruit présent dans le vecteur y par le mauvais conditionnement de la matrice A. Ainsi, même de faibles perturbations peuvent entraîner des erreurs importantes sur la solution estimée.

L'utilisation de la factorisation QR permet d'améliorer la stabilité numérique par rapport aux équations normales, mais elle ne permet pas de réduire la sensibilité intrinsèque du problème aux perturbations.

Pour limiter l'effet du bruit sur la solution, il est nécessaire d'introduire des techniques permettant de contrôler l'influence des directions associées aux petites valeurs singulières.

Plusieurs approches peuvent être envisagées :

- **Décomposition en valeurs singulières (SVD)** : elle permet d'analyser la structure du problème et d'identifier les directions responsables de l'amplification du bruit.
- **Troncature des petites valeurs singulières (Truncated SVD)** : en négligeant les valeurs singulières trop faibles, on réduit l'impact du bruit sur la solution en éliminant les directions instables.
- **Régularisation (Tikhonov / Ridge)** :

  $$\beta_\lambda = (A^T A + \lambda I)^{-1} A^T y$$

  cette méthode permet de stabiliser la solution en limitant l'amplification des perturbations présentes dans y.
- **Moindres carrés pondérés** : en introduisant une pondération des observations, il est possible de réduire l'influence des données fortement bruitées sur l'estimation finale.

Ainsi, dans un problème mal conditionné, il ne suffit pas de choisir une méthode de résolution connue Stable (c'est QR dans notre expérience): il est nécessaire de modifier le problème lui-même afin de réduire l'effet du bruit et d'obtenir une solution numériquement stable.

6.2 Instabilité numérique due à l'algorithme : Gram-Schmidt et équations semi-normales

Problème observé

Dans cette expérience, on compare trois méthodes de résolution du problème des moindres carrés : la factorisation QR (Householder), la méthode de Gram-Schmidt classique et les équations semi-normales.

Les résultats montrent une différence significative entre les solutions obtenues :

- la méthode QR (Householder) fournit une solution stable et proche de la solution exacte,

- la méthode de Gram-Schmidt présente des résultats fortement instables,
- la méthode des équations semi-normales donne des résultats fortement instables.

Ces différences sont clairement visibles dans les figures suivantes :

```
 " i    Exact       QR         Semi         GS"
 "-------------------------------------------"
 1    1.0000    1.0000    5.72e+00  -526208.9998
 2    2.0000    2.0000   -3.06e+00  563449.1108
 3    3.0000    3.0000   -1.83e+00  537938.4414
 4    4.0000    4.0000    3.31e+00  76394.7070
 5    5.0000    5.0000    7.50e+00  -278353.8666
 6    6.0000    6.0000    0.00e+00  668241.5834
 7    7.0000    7.0000    6.70e+00  32906.9774
 8    8.0000    8.0000    1.18e+01  -421319.5509
 9    9.0000    9.0000    1.37e+01  -519160.3367
10   10.0000   10.0000    1.04e+01  -42097.3868
 "-------------------------------------------"
 "Erreurs : [QR   Semi   GS]"
  0.0000248   12.257098   1365104.9
```

Figure 6.4: Comparaison QR / Gram-Schmidt / semi-normales — erreurs

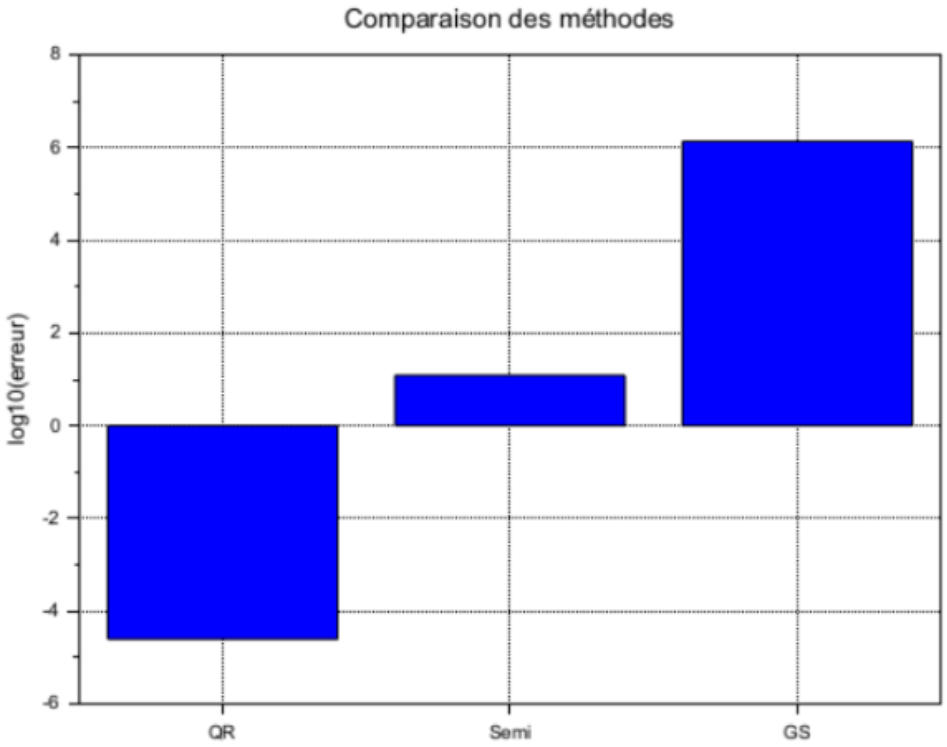


Figure 6.5: Comparaison QR / Gram-Schmidt / semi-normales — erreurs

Origine numérique

L'instabilité observée dans cette expérience provient de deux sources distinctes : la perte d'orthogonalité dans la méthode de Gram-Schmidt classique et l'amplification du conditionnement dans les équations semi-normales.

1. Méthode de Gram-Schmidt classique

La méthode de Gram-Schmidt construit la factorisation QR d'une matrice $A = [a_1, \ldots, a_p]$ en orthogonalisant successivement ses colonnes.

Pour chaque colonne a_j, on calcule :

$$r_{ij} = q_i^T a_j, \quad i = 1, \ldots, j-1$$

$$v_j = a_j - \sum_{i=1}^{j-1} r_{ij} q_i$$

$$r_{jj} = \|v_j\|, \quad q_j = \frac{v_j}{r_{jj}}$$

On obtient ainsi une matrice $Q = [q_1, \ldots, q_p]$ supposée orthogonale et une matrice triangulaire supérieure R telle que :

$$A = QR$$

Origine numérique de l'instabilité

En arithmétique flottante, les calculs de produits scalaires et les soustractions introduisent des erreurs d'arrondi. Ces erreurs s'accumulent lors des projections successives.

En particulier, les vecteurs v_j sont obtenus par soustraction de vecteurs presque colinéaires lorsque la matrice A est mal conditionnée. Cela entraîne des phénomènes d'annulation numérique, rendant les vecteurs v_j très sensibles aux erreurs.

Par conséquent, les vecteurs q_j ne sont plus parfaitement orthogonaux, et la matrice Q vérifie en pratique :

$$Q^T Q \neq I$$

Cette perte d'orthogonalité dégrade la qualité de la factorisation QR et entraîne une erreur dans la solution du problème des moindres carrés.

Impact sur le calcul de la solution

La solution est calculée à partir de :

$$\beta = R^{-1} Q^T y$$

Cette formule suppose que Q est orthogonale. Lorsque ce n'est pas le cas, la transformation du problème n'est plus correcte, ce qui introduit une erreur significative sur β.

Différence avec la méthode de Householder

Contrairement à Gram-Schmidt, la méthode de Householder construit Q comme produit de matrices orthogonales (réflexions). Ces transformations préservent numériquement l'orthogonalité.

Ainsi, la matrice Q obtenue par Householder vérifie très précisément :

$$Q^T Q = I$$

Cette propriété garantit la stabilité numérique de la méthode, contrairement à Gram-Schmidt classique.

Conclusion
L'instabilité de la méthode de Gram-Schmidt classique provient de la perte d'orthogonalité de la matrice Q due aux erreurs d'arrondi, particulièrement en présence d'une matrice mal conditionnée. La méthode de Householder permet d'éviter ce problème en préservant l'orthogonalité de manière numériquement stable.

2. Équations semi-normales

Les équations semi-normales reposent sur la résolution du système :

$$R^T Rx = A^T b$$

où R est issu de la factorisation QR (par méthode de Householder) de A.

Comme la matrice Q est orthogonale, le conditionnement de A est conservé dans R, c'est-à-dire que $\kappa(R) \approx \kappa(A)$. Ainsi, si A est mal conditionnée, R l'est également.

Le passage à $R^T R$ amplifie ce conditionnement :

$$\kappa(R^T R) = \kappa(A)^2$$

Par conséquent, les erreurs d'arrondi et les perturbations dans les données sont fortement amplifiées lors de la résolution. Même en présence d'un bruit très faible, cette amplification peut conduire à des erreurs importantes dans la solution.

Conclusion

L'instabilité observée dans cette expérience est due à l'amplification du conditionnement dans les équations semi-normales. Cet effets sont particulièrement marqués lorsque la matrice A est mal conditionnée.

Solution possible

Une solution consiste à utiliser la factorisation QR basée sur les transformations de Householder. Cette méthode présente plusieurs avantages :

- elle évite la formation de $A^T A$,
- elle préserve l'orthogonalité de la matrice Q,
- elle est numériquement stable.

La méthode QR (Householder) fournit ainsi des solutions fiables pour les problèmes de moindres carrés.

Conclusion

Cette expérience montre que la stabilité numérique dépend fortement de la méthode utilisée :

- Gram-Schmidt classique est instable à cause des erreurs d'arrondi,
- les équations semi-normales amplifient le conditionnement,
- la méthode QR (Householder) est la plus fiable.

Ces résultats illustrent l'importance du choix de l'algorithme dans la résolution des problèmes de moindres carrés.

6.3 Effet du mauvais centrage sur la stabilité numérique

Génération du jeu de données

Afin d'étudier l'impact du mauvais centrage sur la stabilité numérique, nous avons construit une matrice $A \in \mathbb{R}^{100\times10}$.

Une matrice aléatoire initiale a été générée suivant une distribution normale centrée. Ensuite, chaque colonne a été décalée par une constante de très grande amplitude afin de simuler des données fortement mal centrées :

$$A(:, j) = A_0(:, j) + 10^{10} \times j$$

Le vecteur de réponse est construit selon :

$$y = A\beta_{\text{exact}} + \varepsilon$$

où $\beta_{\text{exact}} = (1, \ldots, 10)^T$ et ε est un bruit très faible (10^{-20}), ce qui permet d'isoler les effets purement numériques.

```
"Extrait de A (mal centré) :"
       column 1 to 7
 1.000D+10   2.000D+10   3.000D+10   4.000D+10   5.000D+10   6.000D+10   7.000D+10
 1.000D+10   2.000D+10   3.000D+10   4.000D+10   5.000D+10   6.000D+10   7.000D+10
 1.000D+10   2.000D+10   3.000D+10   4.000D+10   5.000D+10   6.000D+10   7.000D+10
 1.000D+10   2.000D+10   3.000D+10   4.000D+10   5.000D+10   6.000D+10   7.000D+10
 1.000D+10   2.000D+10   3.000D+10   4.000D+10   5.000D+10   6.000D+10   7.000D+10
       column 8 to 10
 8.000D+10   9.000D+10   1.000D+11
 8.000D+10   9.000D+10   1.000D+11
 8.000D+10   9.000D+10   1.000D+11
 8.000D+10   9.000D+10   1.000D+11
 8.000D+10   9.000D+10   1.000D+11
```

Figure 6.6: Extrait de la matrice A avec données fortement mal centrées

Problème observé

Les résultats obtenus montrent que la matrice A est extrêmement mal conditionnée.

```
"conditonnement de A "
 2.430D+11
```

Figure 6.7: Conditionnement de la matrice A

Cette situation entraîne une instabilité dans la résolution du problème des moindres carrés.

On observe que :

- Les équations normales produisent une solution fortement instable.
- La méthode QR (Householder) fournit une solution plus robuste, mais qui présente néanmoins un décalage par rapport à la solution exacte.

```
 " i     Exact        QR          NE
 "-------------------------------------
 1     1.0000     1.0002     0.00e+00
 2     2.0000     2.0000     0.00e+00
 3     3.0000     2.9999     0.00e+00
 4     4.0000     4.0000     0.00e+00
 5     5.0000     5.0000     0.00e+00
 6     6.0000     6.0001     0.00e+00
 7     7.0000     7.0001     0.00e+00
 8     8.0000     7.9998     0.00e+00
 9     9.0000     9.0001     0.00e+00
10    10.0000    10.0000     3.85e+01
```

Figure 6.8: Comparaison des solutions obtenues par QR et équations normales

Origine numérique du problème

L'instabilité observée provient du mauvais centrage des données.

En effet, l'ajout de constantes de très grande amplitude (10^{10}) dans chaque colonne introduit des valeurs dominantes dans la matrice A, ce qui dégrade fortement son conditionnement.

Un conditionnement élevé amplifie les erreurs numériques (bruit ou erreurs d'arrondi), ce qui rend la solution très sensible aux perturbations.

Ainsi, même en présence d'un bruit extrêmement faible, les coefficients estimés peuvent s'écarter significativement de la solution exacte.

Solution possible

Une solution consiste à centrer les données avant de résoudre le problème.

Le centrage est une étape fondamentale en pratique, notamment en data science et en apprentissage statistique. Par exemple, dans des méthodes comme l'Analyse en Composantes Principales (ACP), les données sont systématiquement centrées, et le plus souvent réduites, afin d'éliminer les effets de biais et de garantir une interprétation pertinente des résultats.

Cette pratique permet d'améliorer le conditionnement des matrices manipulées et d'assurer une meilleure stabilité numérique des algorithmes.

Le centrage est effectué en soustrayant la moyenne de chaque colonne :

$$A_c(:, j) = A(:, j) - \text{mean}(A(:, j))$$

Cette opération supprime le biais introduit par les grandes constantes et améliore significativement le conditionnement de la matrice.

```
 "-------------------------------------------"
 " i    Exact       QR         NE         Centré"
 "-------------------------------------------"
 1    1.0000    1.0002    0.00e+00    1.0000
 2    2.0000    2.0000    0.00e+00    2.0000
 3    3.0000    2.9999    0.00e+00    3.0000
 4    4.0000    4.0000    0.00e+00    4.0000
 5    5.0000    5.0000    0.00e+00    5.0000
 6    6.0000    6.0001    0.00e+00    6.0000
 7    7.0000    7.0001    0.00e+00    7.0000
 8    8.0000    7.9998    0.00e+00    8.0000
 9    9.0000    9.0001    0.00e+00    9.0000
10   10.0000   10.0000    3.85e+01   10.0000
```

Figure 6.9: Comparaison des solutions : QR, équations normales et données centrées

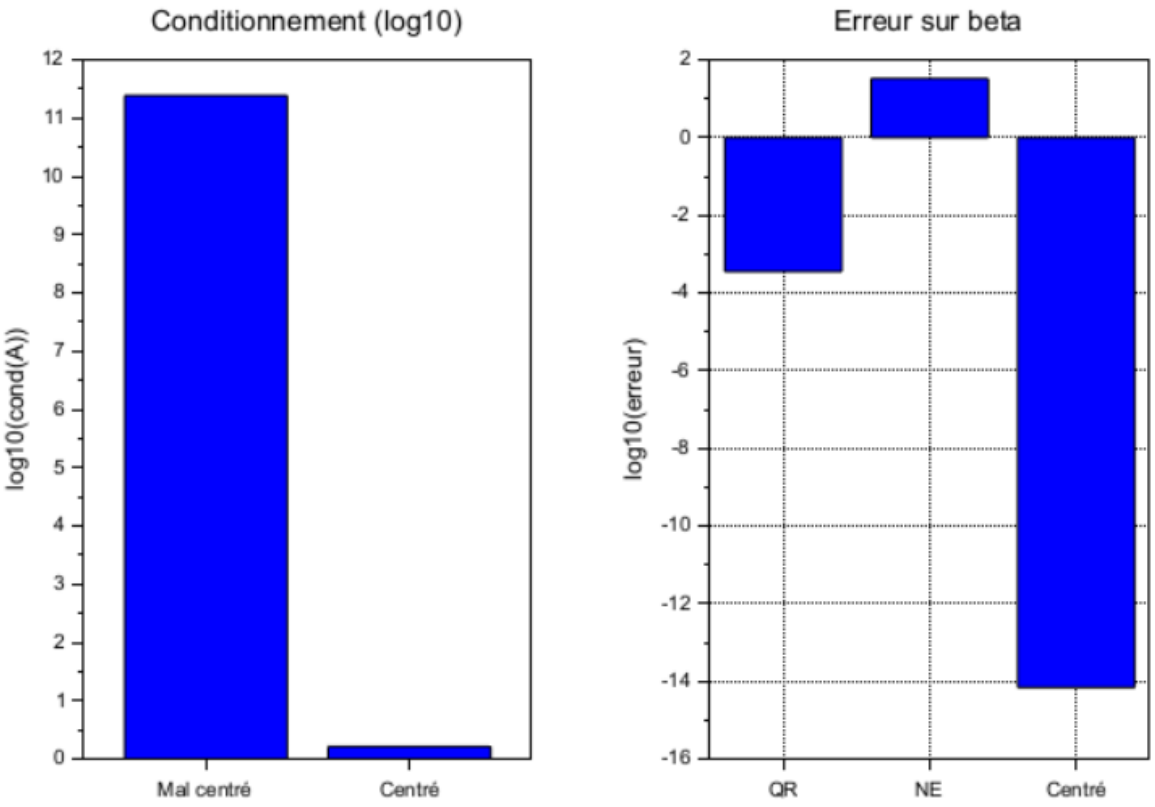


Figure 6.10: Comparaison du conditionnement et de l'erreur avant et après centrage

Après centrage :

- Le conditionnement diminue fortement.
- La solution obtenue est beaucoup plus proche de la solution exacte.
- La stabilité numérique est améliorée.

Conclusion : Le mauvais centrage des données, en introduisant des valeurs de grande amplitude, dégrade fortement le conditionnement de la matrice et rend le problème numériquement instable. Le centrage constitue une étape essentielle pour obtenir des solutions fiables.
En plus du centrage , en pratique les données sont souvent aussi réduites (pratique en ACP) pour eviter tout risque d'instabilité qui peut apparaître suite a des données utilisant des échelles différentes .

Chapitre 7

Analyse critique et conclusion

L'ensemble des expériences menées dans ce projet met en évidence un point fondamental : lorsque le conditionnement de la matrice A est extrêmement élevé, le problème devient intrinsèquement instable, indépendamment de l'algorithme utilisé.

En effet, la théorie des perturbations montre que l'erreur relative sur la solution vérifie une relation du type :

$$\frac{\|\hat{\beta} - \beta\|}{\|\beta\|} \approx \kappa(A) \, \frac{\|\delta y\|}{\|y\|}$$

où $\kappa(A)$ est le conditionnement de la matrice A. Ainsi, lorsque $\kappa(A)$ est très grand, même une perturbation très faible sur les données entraîne une erreur importante sur la solution.

Par conséquent, dans les cas extrêmes, le problème ne dépend plus uniquement de l'algorithme, mais doit être traité à la source, notamment par un prétraitement des données (centrage, réduction, filtrage éventuel du bruit).

Quelle formulation est la plus fiable numériquement ?

Parmi les différentes formulations étudiées, la résolution par **décomposition QR (transformations de Householder)** apparaît comme la méthode la plus fiable numériquement.

Les équations normales reposent sur la résolution du système :

$$A^T A \, \beta = A^T y$$

ce qui implique un conditionnement vérifiant :

$$\kappa(A^T A) = \kappa(A)^2$$

Cette élévation au carré amplifie considérablement les erreurs numériques, rendant cette approche particulièrement instable, comme observé dans l'ensemble des expériences.

Les équations semi-normales, qui reposent sur :

$$R^T R \, \beta = A^T y$$

présentent le même défaut, puisque :

$$\kappa(R^T R) = \kappa(A)^2$$

De même, la méthode de Gram-Schmidt classique introduit une instabilité supplémentaire due à la perte d'orthogonalité de la matrice Q, ce qui dégrade la qualité de la factorisation QR obtenue.

En revanche, la méthode QR par transformations de Householder repose sur la résolution du système :

$$R\beta = Q^T y$$

sans jamais former $A^T A$. De plus, les transformations de Householder préservent numériquement l'orthogonalité de Q, ce qui garantit une *stabilité* de l'algorithme.

Ainsi, contrairement aux autres méthodes, la méthode QR n'amplifie pas le conditionnement du problème et fournit la solution la plus fiable dans l'ensemble des cas étudiés.

Ces résultats ont été confirmés expérimentalement dans ce projet, à travers la comparaison avec les équations normales, les équations semi-normales, la méthode de Gram-Schmidt et la résolution directe par factorisation LU, toutes montrant une sensibilité accrue aux erreurs numériques.

Pourquoi les équations normales sont-elles dangereuses ?

Les équations normales reposent sur la résolution du système :

$$A^T A\,\beta = A^T y$$

Cette formulation présente un défaut structurel majeur : elle remplace la matrice A par la matrice $A^T A$, ce qui entraîne une amplification du conditionnement :

$$\kappa(A^T A) = \kappa(A)^2$$

Comme le montre l'analyse du problème des moindres carrés, cette amplification a un impact direct sur l'erreur de la solution. En particulier, la borne d'erreur obtenue pour les équations normales est de la forme :

$$\frac{\|\hat{\beta} - \beta\|}{\|\beta\|} \lesssim c\,\kappa(A)^2\,u$$

où u est la précision machine.

Ainsi, contrairement aux méthodes basées sur la factorisation QR, dont l'erreur est proportionnelle à $\kappa(A)$, les équations normales introduisent une dépendance en $\kappa(A)^2$, ce qui entraîne une dégradation très importante de la précision lorsque la matrice est mal conditionnée.

De plus, Comme le montre l'analyse présentée dans le chapitre de référence sur les problèmes de moindres carrés, la méthode des équations normales n'est pas *backward stable*, contrairement à la factorisation QR par transformations de Householder.Cela signifie que la solution calculée ne correspond pas nécessairement à la solution exacte d'un problème légèrement perturbé, ce qui renforce son instabilité numérique.

Un second aspect critique est que cette instabilité est souvent masquée par le résidu. En effet, les équations normales minimisent directement $\|A\beta - y\|$, ce qui garantit un résidu faible, même lorsque la solution β est très éloignée de la solution exacte.

Ainsi, les équations normales cumulent deux défauts majeurs :

- une amplification quadratique du conditionnement, entraînant une forte sensibilité aux erreurs numériques ;
- une absence de stabilité backward, combinée à un résidu trompeur qui peut masquer des erreurs importantes sur la solution.

Conclusion : les équations normales sont dangereuses car elles dégradent intrinsèquement le problème numérique et peuvent fournir des solutions très inexactes tout en donnant l'illusion d'un bon ajustement.

Peut-on faire confiance à un algorithme « qui converge » ?

Les expériences menées apportent une réponse claire : **non, pas nécessairement**.

Dans l'ensemble des cas étudiés, les algorithmes ont convergé, c'est-à-dire qu'ils ont produit une solution avec un résidu faible, parfois très proche du niveau de bruit introduit dans les données.

Cependant, cette convergence ne garantit pas la qualité de la solution. Dans plusieurs expériences, les coefficients estimés β étaient très éloignés des valeurs exactes, malgré un ajustement apparemment excellent.

Ce phénomène s'explique par le mauvais conditionnement du problème : un algorithme peut trouver un minimiseur du résidu sans que ce minimiseur soit proche de la solution exacte. Ainsi, la convergence reflète uniquement la résolution du problème numérique posé, et non la fiabilité du modèle obtenu.

Lien avec les pratiques réelles en data science

Les phénomènes étudiés dans ce projet correspondent directement à des problématiques centrales en data science, en particulier dans les tâches de modélisation statistique et d'apprentissage supervisé.

Tout d'abord, la **multicolinéarité**, analysée dans les premières parties, est très fréquente dans les jeux de données réels. En pratique, plusieurs variables explicatives peuvent contenir une information redondante ou fortement corrélée. Par exemple, dans des données socio-économiques, des variables comme le revenu, les dépenses et le niveau de vie sont souvent liées. Nos expériences montrent que, dans ce cas, un modèle peut présenter un excellent ajustement (faible résidu), tout en produisant des coefficients instables et difficilement interprétables.

Ensuite, le **mauvais centrage et les différences d'échelle**, étudiés dans nos expériences, constituent un problème fondamental en data science. Les jeux de données combinent fréquemment des variables de nature et d'amplitude très différentes. Cette hétérogénéité dégrade le conditionnement des matrices utilisées dans les algorithmes d'estimation, ce qui peut rendre les résultats numériquement instables. C'est pour cette raison que des étapes de *standardisation* (centrage et réduction) sont systématiquement appliquées avant l'entraînement de nombreux modèles.D'ailleurs, en ACP (analyse en composantes principales) on utilise souvent des données centrées réduites pour analyser nos données .

Enfin, les situations où la matrice des données est mal conditionnée, notamment celles construites via la SVD dans ce projet, correspondent à des cas où certaines directions de variation sont très faiblement informatives. En data science, cela se traduit par la présence de variables redondantes ou peu pertinentes, rendant les modèles très sensibles au bruit.

Ces observations mettent en évidence un point essentiel : en data science, la qualité des résultats ne dépend pas uniquement de l'algorithme utilisé, mais aussi de la structure des données. Par conséquent, le prétraitement des données centrage, mise à l'échelle et analyse du conditionnement constitue une étape indispensable pour garantir la stabilité numérique et la fiabilité des modèles.

Conclusion générale

Ce projet met en évidence un point essentiel : un algorithme peut être mathématiquement correct sans être numériquement fiable. En pratique, le conditionnement du problème joue un rôle déterminant sur la qualité des résultats.

La décomposition QR apparaît comme la méthode la plus stable pour la résolution des moindres carrés, tandis que les équations normales peuvent conduire à des erreurs importantes, malgré un bon ajustement apparent.

Ces observations montrent que, en data science, il ne suffit pas d'obtenir un bon résidu pour valider un modèle. Il est indispensable d'analyser le conditionnement, de choisir une formulation adaptée et de préparer les données afin de garantir des résultats fiables.

Chapitre 8

Notes et Références

Les analyses théoriques présentées dans ce projet, en particulier celles relatives à l'origine numérique des instabilités (conditionnement, amplification des erreurs, stabilité des méthodes), s'appuient sur des résultats classiques issus de la littérature en analyse numérique.

Plus précisément, une partie des développements s'inspire du chapitre consacré au problème des moindres carrés dans l'ouvrage de référence, qui traite des questions de conditionnement, de stabilité numérique et de comparaison entre les différentes méthodes de résolution.

Les documents suivants ont été utilisés comme support théorique :

- Chapitre de référence sur les moindres carrés : Lien vers le chapitre
- Ouvrage complet : Lien vers le livre

Ces références ont permis de relier les observations expérimentales à des résultats théoriques établis, notamment en ce qui concerne la stabilité backward des algorithmes et l'impact du conditionnement sur la précision des solutions.